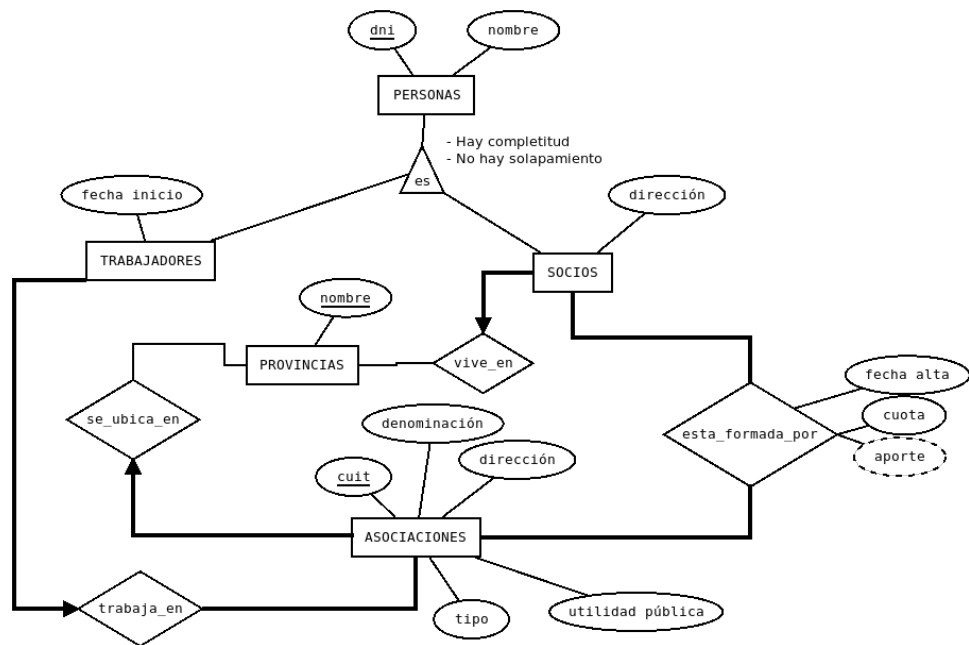


1) (4 puntos) Dado el siguiente DER:



El diagrama corresponde a la base de datos del subsistema de gestión de socios y trabajadores de ONGs. De los socios y trabajadores se guarda su DNI y nombre completo, además de los primeros también se registran datos de domicilio y de los segundos la fecha en que empezaron a trabajar (trabajan en una única asociación). Los socios pueden serlo de varias asociaciones (al menos una) y se registra la fecha de alta, la cuota mensual y el aporte anual (monto de cuota por meses del año). Los socios no pueden trabajar en las asociaciones.

Se pide:

a) Escriba las **sentencias SQL** para crear el **esquema relacional** correspondiente utilizando las **cláusulas** y **parámetros** que considere necesarios para responder de la manera más adecuada a las siguientes consignas:

i. Escribir como un comentario fecha y hora en que está realizando la tarea;

-- Tarea realizada el 29/09/2022 a partir de las 20,15hs

ii. Crear la base de datos *Primer\_parcial\_T2*;

CREATE DATABASE Primer\_parcial\_T2;

iii. Crear las tablas especificando las restricciones de clave (primarias, candidatas), clave foránea (y manejo de integridad referencial), dominio, campos no nulos, y demás opciones que correspondan para capturar la semántica del DER;

/\* Lo correcto en este punto sería tener en cuenta también cómo prevenir la creación de claves foráneas que referencien a tablas que aún no se crearon. También considerar que la jerarquía se podría pasar a 3 tablas, pero para este caso particular es más conveniente pasar a 2 \*/

```
CREATE TABLE PROVINCIAS (  
    nombre CHAR(20),  
    PRIMARY KEY (nombre)  
);  
  
CREATE TABLE ASOCIACIONES (  
    cuit BIGINT,  
    denominacion VARCHAR(25) NOT NULL,  
    utilidad_publica VARCHAR(25),  
    tipo CHAR(20) NOT NULL,  
    direccion TEXT,  
    provincia CHAR(20) NOT NULL,  
    PRIMARY KEY (cuit),  
    FOREIGN KEY (provincia) REFERENCES PROVINCIAS(nombre)  
        ON DELETE NO ACTION ON UPDATE CASCADE  
);  
  
CREATE TABLE TRABAJADORES (  
    dni INT(8),  
    nombre VARCHAR(50) NOT NULL,  
    fecha_inicio DATE DEFAULT (CURRENT_DATE),  
    asociacion BIGINT NOT NULL,  
    PRIMARY KEY (dni),  
    FOREIGN KEY (asociacion) REFERENCES ASOCIACIONES(cuit)  
        ON DELETE NO ACTION ON UPDATE NO ACTION  
);  
  
CREATE TABLE SOCIOS (  
    dni INT(8),  
    nombre VARCHAR(50) NOT NULL,  
    direccion VARCHAR(50) NOT NULL,  
    fecha_alta DATE NOT NULL,  
    cuota BIGINT NOT NULL,  
    aporte BIGINT NOT NULL,  
    PRIMARY KEY (dni),  
    FOREIGN KEY (dni) REFERENCES PERSONAS(dni)  
        ON DELETE NO ACTION ON UPDATE NO ACTION,  
    FOREIGN KEY (asociacion) REFERENCES ASOCIACIONES(cuit)  
        ON DELETE NO ACTION ON UPDATE NO ACTION
```

```
        dni INT(8),
        nombre VARCHAR(50) NOT NULL,
        dirección VARCHAR(100),
        provincia CHAR(20) NOT NULL,
        PRIMARY KEY (dni),
        FOREIGN KEY (provincia) REFERENCES PROVINCIAS(nombre)
            ON DELETE NO ACTION ON UPDATE CASCADE
    );

CREATE TABLE esta_formada_por (
    socio INT(8),
    asociacion BIGINT,
    fecha_alta DATE DEFAULT (CURDATE()),
    cuota DECIMAL(7,2) NOT NULL,
    aporte_anual DOUBLE AS (cuota * 12),
    PRIMARY KEY (socio, asociacion),
    FOREIGN KEY (socio) REFERENCES SOCIOS(dni)
        ON UPDATE CASCADE ON DELETE NO ACTION,
    FOREIGN KEY (asociacion) REFERENCES ASOCIACIONES(cuit)
        ON UPDATE CASCADE ON DELETE CASCADE
    );
```

iv. Crear un índice que permita optimizar búsquedas por la denominación de las asociaciones;

```
CREATE INDEX por_nombre ON ASOCIACIONES (denominacion);
```

v. Se decide que cada asociación debe tener un trabajador asignado como coordinador general y debe registrarse además la fecha desde la cual cumple esa función. Haga las modificaciones necesarias de acuerdo a lo ya desarrollado en el ítem iii para reflejar esta nueva situación;

```
ALTER TABLE ASOCIACIONES
    ADD COLUMN coordinador INT NOT NULL,
    ADD COLUMN coordina_desde DATE NOT NULL DEFAULT (CURRENT_DATE),
    ADD FOREIGN KEY (coordinador) REFERENCES TRABAJADORES(dni)
        ON UPDATE CASCADE ON DELETE NO ACTION
;
```

b) Realizar las siguientes acciones de LMD, **comentando** previamente si es necesario tener en cuenta alguna **consideración** o realizar **acciones adicionales** (escriba las sentencias SQL que las ejecutan):

i. Registrar un nuevo trabajador con los siguientes datos: **DNI:** 37222333, **NOMBRE:** ‘Martín Peralta’, trabaja en la asociación ‘Capacitaciones Ad-Hoc’ desde el 01/08/2018;

```
/* Tener en cuenta que previamente deben estar cargados los registros de la asociación. También que los tipos,
longitudes, y obligatoriedad de datos definidos en los campos de tabla permitan cargar los valores pedidos. También
en este caso se debería resolver el coordinador, al ser obligatorio */
INSERT INTO TRABAJADORES(dni, nombre, asociacion)
SELECT 37222333, ‘Martín Peralta’, a.cuit
FROM ASOCIACIONES a
WHERE a.denominacion LIKE 'Capacitaciones Ad-Hoc';
```

ii. El trabajador Gustavo Gómez decide dejar de cumplir funciones de coordinador general, por lo que para reemplazarlo se designa temporalmente a Martín Peralta;

```
/* Previamente debe estar cargado el registro del viejo coordinador */
/* Se pasan las coordinaciones de los proyectos a cargo de Gustavo Gómez a Martín Peralta */
UPDATE ASOCIACIONES a, TRABAJADORES t
SET a.coordinador = t.dni, a.coordina_desde = curdate()
WHERE t.nombre LIKE 'Martin Peralta' AND p.coordinador = (
    SELECT dni
    FROM TRABAJADORES
    WHERE nombre LIKE 'Gustavo Gómez'
);
/* Se refleja la situación de Gustavo Gómez */
DELETE FROM TRABAJADORES
WHERE nombre LIKE 'Gustavo Gómez';
```

iii. Eliminar de la base de datos los socios que viven en Tucumán;

```
DELETE FROM esta_formada_por fp
WHERE fp.socio in (
    SELECT dni
```

```
FROM SOCIOS
WHERE provincia LIKE 'TUCUMÁN');
DELETE FROM SOCIOS
WHERE provincia LIKE 'TUCUMÁN';
```

2) (4 puntos) Dado el escenario siguiente resolver usando lenguaje SQL:

**Alumnos**(numa: integer, nombrea: string, carrera: string, nivel: string, edad: integer)

**Cursos**(idc: integer, nombre: string, horarios: string, aula: string, idp: integer)

CF: idp → Profesor(idp)

**Matriculado**(numa: integer, curso: string, nota: real)

CF: numa → Alumnos(numa); curso → Cursos(idc)

**Profesores**(idp: integer, nombrep: string, iddep: integer)

/\* Hay varias alternativas para cada consulta. Destacar si se usa algo de lo aprendido (p.e. CTE, TDL) \*/

- a) Averiguar el nombre de todos los cursos que se dictan en el aula PA-10 o tienen matriculados cinco o más alumnos.

```
SELECT c.nombre
FROM Cursos c
WHERE c.aula LIKE 'PA-10' OR c.idc IN (
    SELECT m.curso
    FROM Matriculado m
    GROUP BY m.curso
    HAVING count(*) >= 5
);
```

- b) Averiguar si hay algún curso en el que se hayan matriculado todos los alumnos de la carrera “Ciencias Políticas”.

```
SELECT c.nombre
FROM Cursos c
WHERE NOT EXISTS (
    SELECT *
    FROM Alumnos a
    WHERE a.carrera LIKE 'Ciencias Políticas' AND NOT EXISTS (
        SELECT *
        FROM Matriculado m
        WHERE m.curso = c.idc AND m.numa = a.numa
    ));
```

- c) Crear una vista que muestre el nombre de los alumnos matriculados en el mayor número de cursos.

```
CREATE VIEW mas_matriculados AS
WITH conteo AS (
    SELECT numa, COUNT(*) tot_c
    FROM Matriculado
    GROUP BY numa)
SELECT a.nombrea
FROM Alumnos a, conteo
WHERE a.numa = conteo.numa AND conteo.tot_c = (
    SELECT MAX(tot_c)
    FROM conteo)
;
```